Software Architecture and Design Specification

Project: Online Quiz System (console-based MCQ quiz, C++17)

Version: 2.0

Authors: Debangana, Chinmay, Satvik, Sunil

Date: 06-11-2026

Status: Final for review

Revision History

Version	Date	Author	Change Summary
1.0	28-09-2026	Team	Baseline architecture
1.1	03-11-2026	Team	Added Question Loader and topic-based quiz flow
2.0	06-11-2026	Team	Aligned with the implemented modules (commit 05bc7d4); UML component, class and sequence diagrams; API, error handling and UX sections; summary of changes from v1.0

Approvals

Role	Name	Signature/Date
Course Coordinator		
Project Guide		
Team Lead		

1. Introduction

1.1 Purpose: This document describes the architecture and design of the Online Quiz System as implemented, and shows how each requirement of SRS v2.0 is realised.

1.2 Scope: The console quiz (menus, topic selection, randomised attempts, scoring, retry) and the administrator functions (login, add, modify, delete, view questions).

1.3 Audience: Developers, testers, the course instructor and evaluators, future maintainers.

1.4 Definitions: MCQ, Attempt (five questions on one topic), Question bank (data/questions.txt), Callback (function passed to another component), SD-n (sequence diagram n).

2. Document Overview

2.1 How to use this document: Section 3 gives the architecture: components, pattern, technology, risks, traceability and security. Section 4 gives the design: classes, sequence diagrams, APIs, error handling and UX.

2.2 Related Documents: SRS v2.0, STP v2.0, project README.md and docs/deviations.md.

3. Architecture

3.1 Goals & Constraints

Goals: a clear console flow; correct and repeatable scoring; varied quizzes by topic; question management protected by login; components that can be tested without a real console.

Constraints: C++17 standard library only; console only; single user; one local text file for data; academic timeline.

3.2 Stakeholders & Concerns

Stakeholder	**Concerns**	**Addressed by**
Player	Clear prompts, fair score, variety	QuizController, QuizLogic; 4.5
Administrator	Easy and safe question maintenance	Admin Module, AdminValidation; 3.9
Developer / Tester	Modularity, testability	Injected loader, admin callback and I/O streams (3.5)
Instructor	Requirements traced to design	3.8, SRS section 8

3.3 Component (UML) Diagram

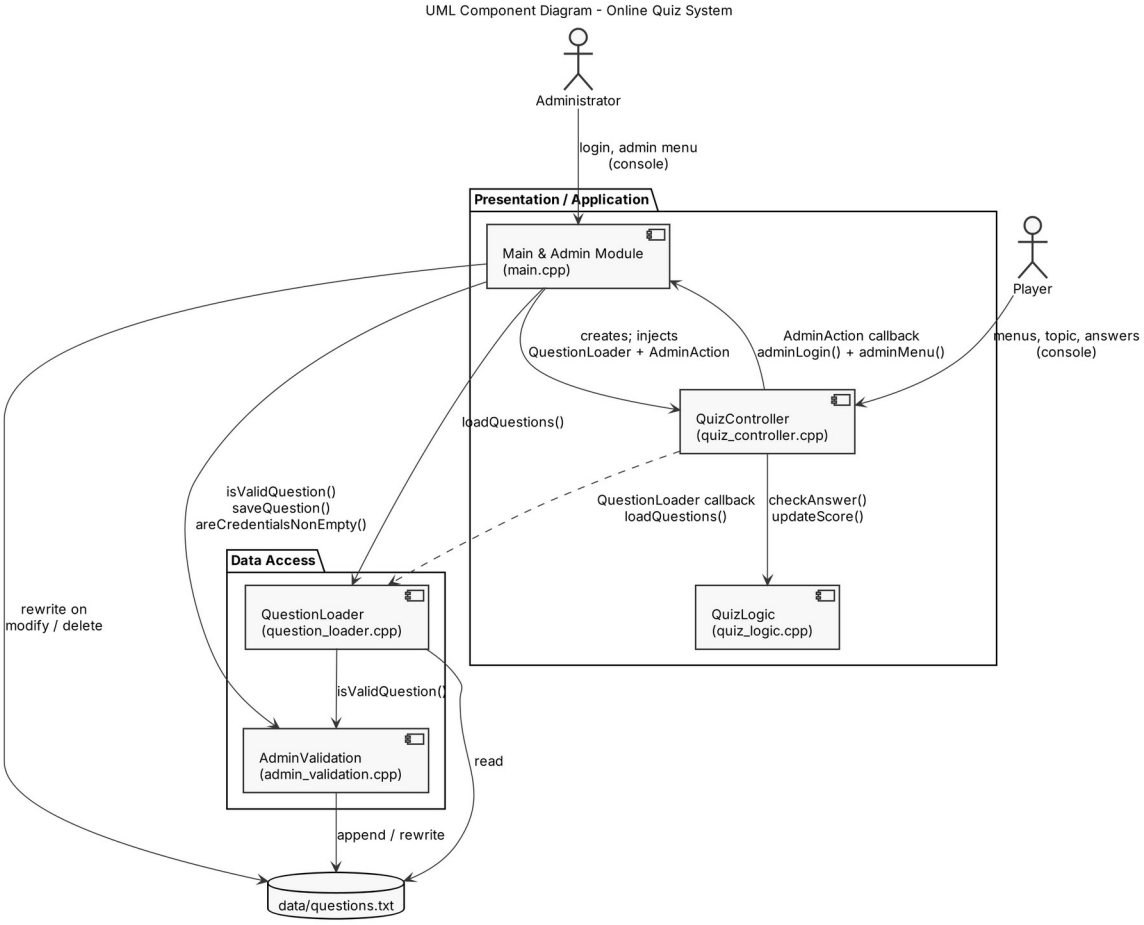


Figure 3.1 - UML component diagram (arrows: calling component to used component)

3.4 Component Descriptions

Component	File	Layer	Responsibility	Requirements
Admin Module (main.cpp)	src/main.cpp	Presentation / Application	Program entry: creates QuizController with the question loader and the admin callback. Admin login with masked password; admin menu (add, modify, delete, view, log out); rewrites the bank on modify/delete.	F-012, F-013, F-014, F-015, NF-003, NF-005, SR-001, SR-002, SR-003, SR-005
QuizController	src/ quiz_controller.cpp	Presentation / Application	Main menu, topic menu, random selection of five questions (std::mt19937, avoids the previous set), question display, answer reading and validation, feedback, final score, result menu. Uses injected istream/ostream.	F-001, F-002, F-003, F-004, F-005, F-006, F-009, F-011, F-015, F-016, F-019, F-020, F-021, F-022, NF-001, NF-003, NF-004, NF-005, SR-004
QuizLogic	src/quiz_logic.cpp	Application	Checks an answer against the key (case-insensitive) and keeps the score; reset at the start of each attempt.	F-006, F-007, F-008, F-010, F-016, NF-002, NF-005
QuestionLoader	src/ question_loader.cpp	Data access	Reads data/questions.txt, splits records on "|", checks field count, topic and answer, and returns valid questions; can filter by topic.	F-017, F-018, F-019, NF-005

Component	File	Layer	Responsibility	Requirements
AdminValidation	src/ admin_validation.cpp	Data access	Shared validation of a question; append, modify and delete a record in the file; non-empty credential check.	F-012, F-013, F-018, NF-005, SR-002, SR-003
Question bank	data/questions.txt	Data	Plain-text question bank: 15 questions for each of ML, DBMS, SE, DSA, GK.	F-017

3.5 Chosen Architecture Pattern and Rationale

Layered architecture with a controller and dependency injection. QuizController runs the user flow and handles console I/O; QuizLogic holds the scoring rules; QuestionLoader and AdminValidation are the only code that reads or writes the question file. main.cpp wires the parts together: it passes QuizController a QuestionLoader callback (load the bank) and an AdminAction callback (login and admin menu), and QuizController takes its input and output streams as parameters.

Rationale: the controller can be unit-tested with scripted input and a fake question list (tests/test_main.cpp); admin code is kept out of the player flow, so a normal user has no path to it (OQ-SR-004); the file format is handled in one place (OQ-F-018).

Alternative	Decision	Reason
Single main() with all logic	Rejected	Hard to test and maintain
Separate class for every concern (v1.0 design: InputHandler, Scorer, ConfigLoader, ...)	Simplified	Too many classes for the project size; layering kept
Client-server	Rejected	Network and multi-user are out of scope

3.6 Technology Stack & Data Stores

Item	Choice
Language	C++17 (string, vector, fstream, sstream, random, algorithm, functional)
Compilers	clang++ (macOS), g++ (Linux), MinGW g++ (Windows); flags -std=c++17 -Wall -Wextra
Password input	termios (macOS/Linux) or _getch (Windows) to turn off echo and print "*"
Build	Makefile (make, make test) or the commands in README.md
Tests	C++ assert-based unit tests in tests/; Bash system-test script tests/system/run_system_tests.sh
CI / release	GitHub Actions workflow .github/workflows/release.yml: builds and tests Linux and Windows binaries when a v* tag is pushed
Data store	data/questions.txt, one record per line: TOPIC|question|A|B|C|D|answer; data/questions.backup.txt as a copy

3.7 Risks & Mitigations

Risk	Mitigation
Invalid or unexpected input	Every menu and answer prompt validates input and asks again
Malformed lines in the question file	QuestionLoader skips them with a warning and line number
Fewer than five questions in a topic	Attempt not started; clear message
Same questions on every retry	Shuffle per attempt and compare with the previous set
Lost data after a bad admin edit	Validation before saving; Y/N confirmation on delete; backup file

Risk	Mitigation
Hard-coded test credentials	Documented limitation (SRS 5.1.3); replace before real use

3.8 Traceability to Requirements

Req ID	Requirement	Component(s)	Design section
OQ-F-001	Start quiz	QuizController	4.2 SD-1, 4.3.1
OQ-F-002	One question at a time	QuizController	4.2 SD-1, 4.3.1
OQ-F-003	Four options A-D	QuizController	4.2 SD-1, 4.3.1
OQ-F-004	Accept answer	QuizController	4.2 SD-1, 4.3.1
OQ-F-005	Reject invalid option	QuizController	4.2 SD-1, 4.3.1
OQ-F-006	Check answer	QuizController, QuizLogic	4.2 SD-1, 4.3.1
OQ-F-007	Score +1 if correct	QuizLogic	4.2 SD-1, 4.3.1
OQ-F-008	Score same if wrong	QuizLogic	4.2 SD-1, 4.3.1
OQ-F-009	Next question	QuizController	4.2 SD-1, 4.3.1
OQ-F-010	Final score	QuizLogic	4.2 SD-1, 4.3.1
OQ-F-011	Show final score	QuizController	4.2 SD-1, 4.3.1
OQ-F-012	Admin add question	Admin Module (main.cpp), AdminValidation	4.2 SD-2
OQ-F-013	Admin modify question	Admin Module (main.cpp), AdminValidation	4.2 SD-3
OQ-F-014	Admin delete question	Admin Module (main.cpp)	4.2 SD-3
OQ-F-015	Block unauthenticated users	QuizController, Admin Module (main.cpp)	3.9, SD-2
OQ-F-016	Reset score on new attempt	QuizController, QuizLogic	4.2 SD-1, 4.3.1
OQ-F-017	Topic question bank	QuestionLoader, Question bank	3.6, 4.3.3
OQ-F-018	Question record format	QuestionLoader, AdminValidation	3.6, 4.3.3
OQ-F-019	Topic selection	QuizController, QuestionLoader	4.2 SD-1, 4.3.1
OQ-F-020	Five random questions	QuizController	4.2 SD-1, 4.3.1
OQ-F-021	Avoid repeating previous set	QuizController	4.2 SD-1, 4.3.1
OQ-F-022	Result menu	QuizController	4.2 SD-1, 4.3.1
OQ-NF-001	Response <= 1 s	QuizController	3.5-3.7
OQ-NF-002	Score reliability	QuizLogic	3.5-3.7
OQ-NF-003	Clear messages	QuizController, Admin Module (main.cpp)	3.5-3.7
OQ-NF-004	No exposure of admin data	QuizController	3.5-3.7
OQ-NF-005	Portability	All components	3.5-3.7
OQ-SR-001	Login before management	Admin Module (main.cpp)	3.9, 4.2 SD-2
OQ-SR-002	Reject wrong credentials	Admin Module (main.cpp), AdminValidation	3.9, 4.2 SD-2
OQ-SR-003	Validate admin input	Admin Module (main.cpp), AdminValidation	3.9, 4.2 SD-2
OQ-SR-004	No direct admin access	QuizController	3.9, 4.2 SD-2
OQ-SR-005	Hide credentials	Admin Module (main.cpp)	3.9, 4.2 SD-2

3.9 Security Architecture

Trust boundaries: (1) keyboard input from any user - untrusted, validated at every prompt; (2) the question file - may be edited outside the program, so every record is validated on load. There is no network interface.

Threat	Control	Component	Requirement
Normal user changes questions	Management only inside the admin menu, reached only after login	QuizController, Admin Module	OQ-SR-001, OQ-SR-004, OQ-F-015
Guessing or empty credentials	Empty-field check; exact match required; generic error message	Admin Module, AdminValidation	OQ-SR-002
Shoulder-surfing the password	Echo off and "*" per character; password never printed	Admin Module	OQ-SR-005
Invalid data saved to the bank	isValidQuestion, topic check and "|" check before saving	AdminValidation, Admin Module	OQ-SR-003
Corrupt record breaks the quiz	Malformed records skipped on load	QuestionLoader	OQ-F-018
Answers visible before answering	Correct option shown only after an answer is submitted	QuizController	OQ-NF-004

Known limitation: the test account admin / admin123 is defined in src/main.cpp (SRS 5.1.3).

3.10 Changes from the v1.0 Design

- Planned classes (ConsoleUI, InputHandler, QuizEngine, Scorer, ReviewManager, BankBuilder, ConfigLoader, ResultLogger, IntegrityChecker, AnswerKeyCodec) were merged into the five modules in 3.4.
- Only data/questions.txt is used; no configuration file, results log or answer-key encoding.
- Admin functions are in main.cpp and AdminValidation and are passed to QuizController as a callback.
- Topic selection, random five-question attempts and the result menu were added (SRS OQ-F-017 to OQ-F-022).
- Full list: docs/deviations.md.

4. Design

4.1 Design Overview

One struct holds a question; one class runs the quiz and one keeps the score; the data and admin code are free functions grouped by file.

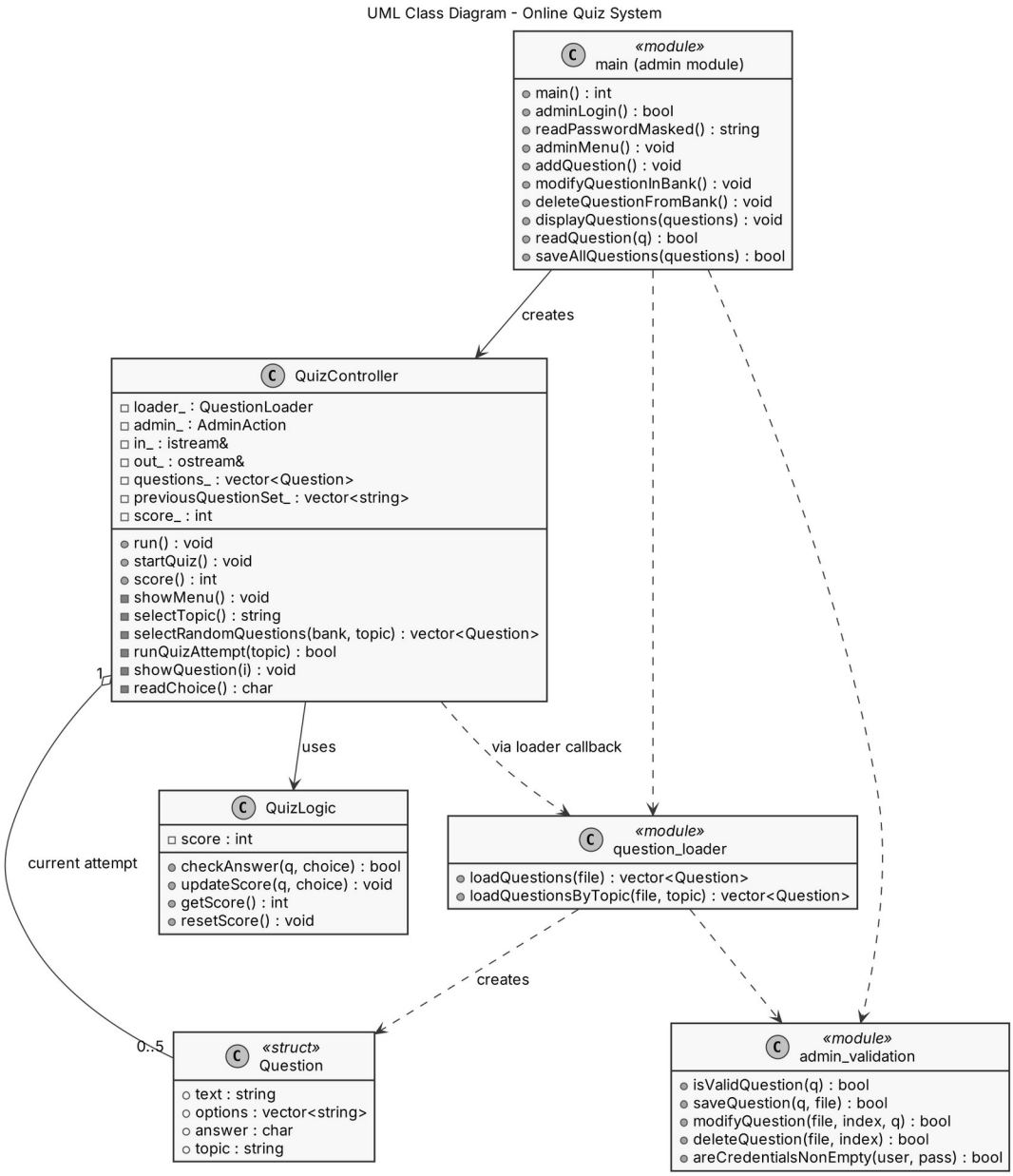


Figure 4.1 - UML class diagram

```
struct Question {
   std::string text;
   std::vector<std::string> options;   // exactly 4 (A-D)
   char answer;                        // A-D
   std::string topic;                  // ML, DBMS, SE, DSA, GK
};
```

4.2 UML Sequence Diagrams

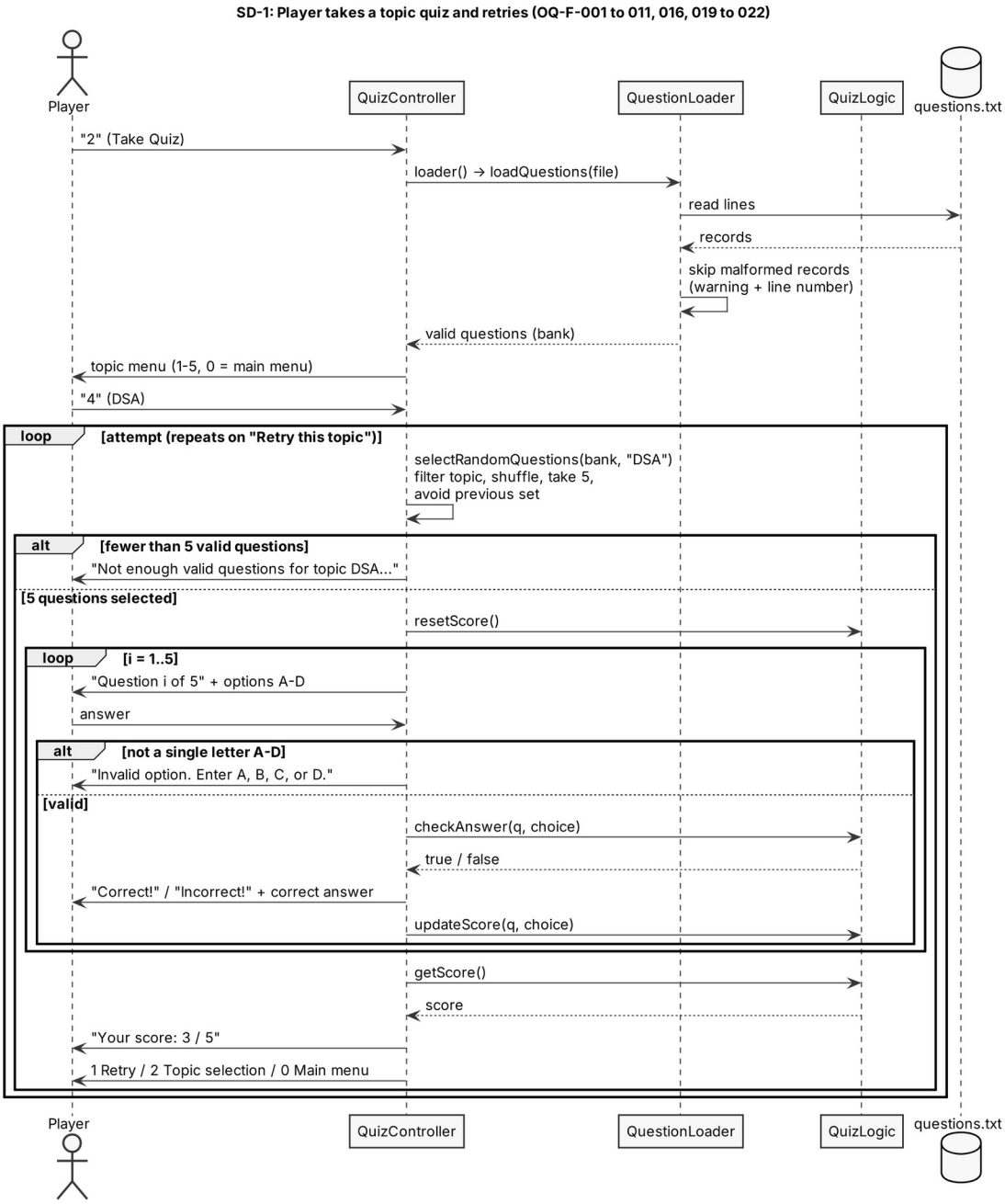


Figure 4.2 - SD-1: Player takes a topic quiz and retries

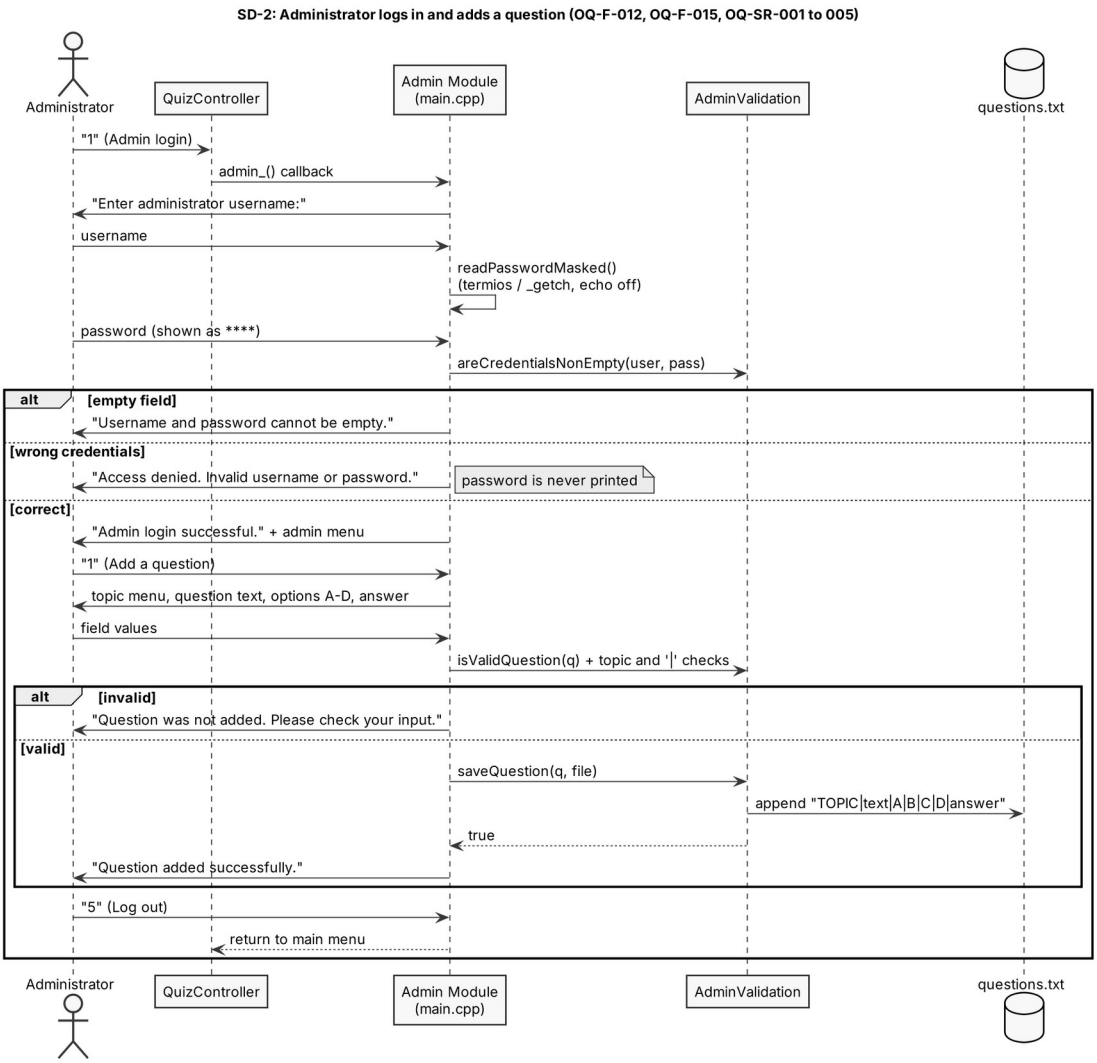


Figure 4.3 - SD-2: Administrator logs in and adds a question

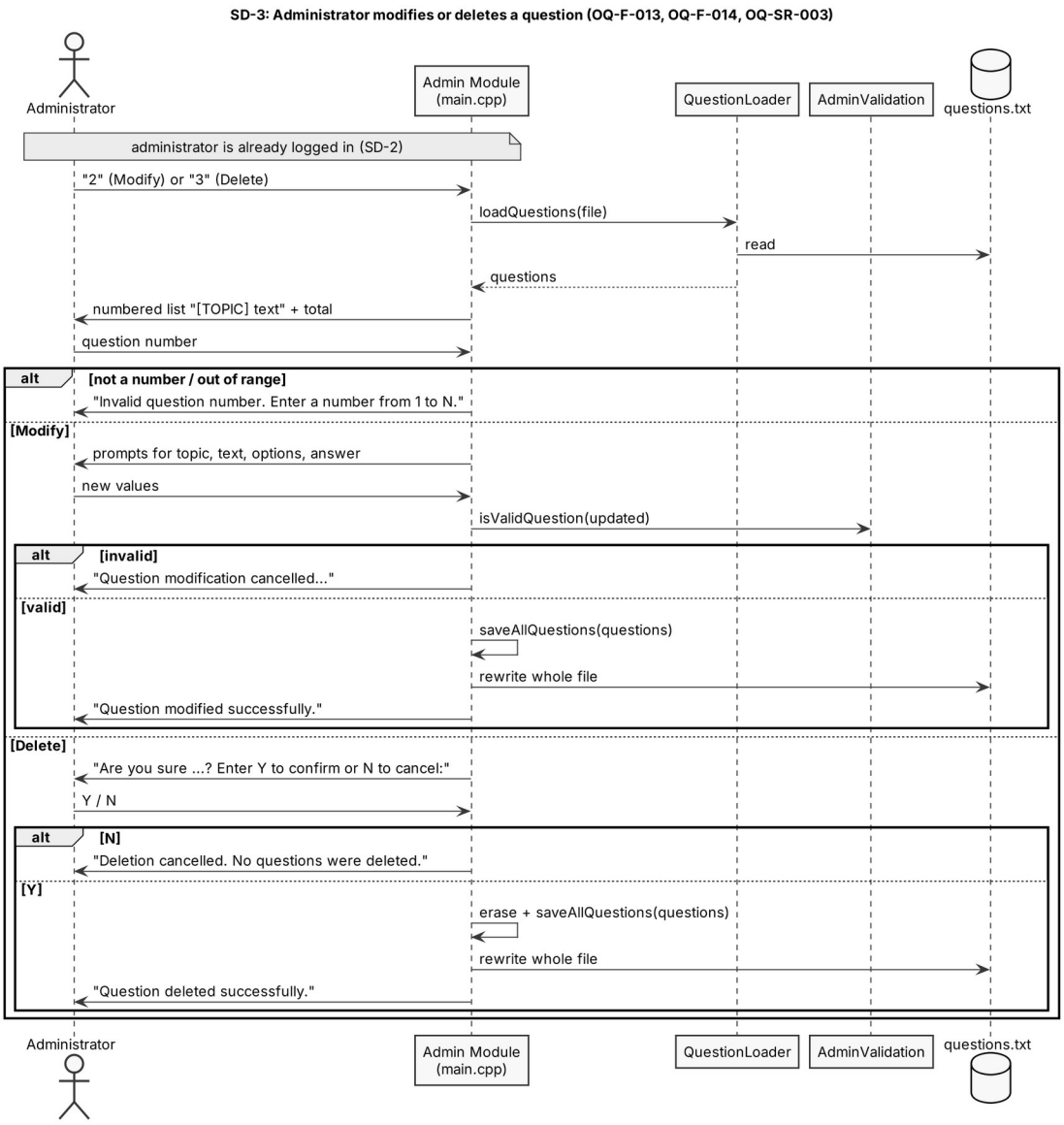


Figure 4.4 - SD-3: Administrator modifies or deletes a question

4.3 API Design

The program is a console application, so its APIs are the C++ interfaces between modules (headers in include/).

4.3.1 QuizController (include/quiz_controller.h)

Function / method	Description	Returns
QuizController(QuestionLoader loader, AdminAction admin, std::istream& in = std::cin, std::ostream& out = std::cout)	Creates the controller with injected loader, admin callback and streams	-
void run()	Main menu loop: 1 admin callback, 2 startQuiz(), 0 exit	-
void startQuiz()	Loads the bank, topic menu, attempts and result menu	-
int score() const	Score of the last attempt	int

Function / method	Description	Returns
selectTopic(), selectRandomQuestions(bank, topic), runQuizAttempt(topic), showQuestion(i), readChoice() (private)	Topic menu; 5 random distinct questions avoiding the previous set; one attempt; display; read A-D	-

4.3.2 QuizLogic (include/quiz_logic.h)

Function / method	Description	Returns
bool checkAnswer(const Question& q, char choice) const	Case-insensitive compare with q.answer	true / false
void updateScore(const Question& q, char choice)	Adds 1 if the answer is correct	-
int getScore() const; void resetScore()	Read / reset the score	int / -

4.3.3 QuestionLoader (include/question_loader.h)

Function / method	Description	Returns
std::vector<Question> loadQuestions(const std::string& filename)	Reads and validates all records; warnings to stderr for skipped lines	valid questions
std::vector<Question> loadQuestionsByTopic(const std::string& filename, const std::string& topic)	Valid questions of one topic	questions

4.3.4 AdminValidation (include/admin_validation.h)

Function / method	Description	Returns
bool isValidQuestion(const Question& q)	Non-empty text, 4 non-empty options, answer A-D	true / false
bool saveQuestion(const Question& q, const std::string& file)	Validates and appends one record	true / false
bool modifyQuestion(const std::string& file, std::size_t index, const Question& q)	Replaces the record at index (used by unit tests; the admin menu uses saveAllQuestions)	true / false
bool deleteQuestion(const std::string& file, std::size_t index)	Removes the record at index (used by unit tests)	true / false
bool areCredentialsNonEmpty(const std::string& user, const std::string& pass)	Both fields non-empty	true / false

4.3.5 Admin Module (src/main.cpp)

Function / method	Description	Returns
bool adminLogin(); std::string readPasswordMasked()	Username prompt, masked password, credential check	true if logged in / password
void adminMenu()	1 add, 2 modify, 3 delete, 4 view, 5 log out	-
bool readQuestion(Question& q); bool readTopic(std::string& t)	Reads and validates the fields of a question	true / false
bool saveAllQuestions(const std::vector<Question>& qs)	Validates every question and rewrites the file	true / false

4.4 Error Handling, Logging & Monitoring

Functions return bool or an empty list instead of throwing; the caller shows a message and returns to a menu, so invalid input never ends the program. End of input (EOF) at any prompt ends the program cleanly.

Situation	**Handling / message**
Invalid main/result menu choice	"Invalid menu choice." and the menu is shown again
Invalid topic	"Invalid topic selection." and the topic menu is shown again
Invalid answer	"Invalid option. Enter A, B, C, or D." - same question
Question file missing	"Error: Unable to open question file: ..." (stderr), then "No valid questions could be loaded."
Malformed record	"Warning: Skipping malformed record at line N..." (stderr); record ignored
Fewer than 5 questions in topic	"Not enough valid questions for topic X. At least five are required."
Empty / wrong credentials	"Username and password cannot be empty." / "Access denied. Invalid username or password."
Invalid question data (admin)	"Question was not added. Please check your input." / "Question modification cancelled..."
Invalid question number	"Invalid question number. Enter a number from 1 to N."
File write failure	"Error: Could not save the question." / "Error: Could not save the changes."

Logging: errors and warnings go to std::cerr; the password is never written anywhere.
Monitoring: not needed for a single-user console program; test results are recorded in docs/test_execution_log.md.

4.5 UX Design

Numbered text menus; every prompt states the valid input; feedback after each answer.

```
=== Online Quiz System ===
1. Admin login
2. Take Quiz
0. Exit
Choose an option: 2

=== Select a Quiz Topic ===
1. Machine Learning (ML)  ...  5. General Knowledge (GK)
0. Return to Main Menu
Choose a topic: 4

Question 1 of 5: Which data structure follows FIFO order?
  A. Stack
  B. Queue
  C. Heap
  D. Graph
Your answer (A-D): b

Correct! Your answer is right.
Correct answer: B. Queue
...
Quiz completed!
Your score: 3 / 5
```

What would you like to do?
1. Retry this topic
2. Return to topic selection
0. Return to main menu

4.6 Open Issues & Next Steps

- Add src/question_loader.cpp to the build commands in .github/workflows/release.yml; without it the next tagged release build will fail to link.
- Replace the hard-coded test credentials with a hashed credential file.
- Update tests/test_question_loader.cpp to the 7-field record format.
- Show "Access denied" instead of "Invalid menu choice." when management is attempted without login (wording only).
- Record the manual checks TC-19, TC-21 (Windows) and TC-26 (masking).

5. Appendices

5.1 Glossary: see SRS 1.4.

5.2 References: SRS v2.0, STP v2.0, ISO/IEC/IEEE 42010, IEEE 1016, project README.md and docs/deviations.md.

5.3 Tools: PlantUML (UML diagrams), clang++/g++, Make, GitHub Actions.